A One-Line-Latency Vision Enhancement Terminal

PYNQ-Z2 | reading aid for the low-vision | 2026 Embedded Chip & System Design Competition, AMD track, self-selected topic (junior group) | Zynq-7000 XC7Z020 | Vivado / Vitis HLS 2026.1

Motivation

Around 17 million people in China live with low vision, and shop-bought electronic magnifiers are expensive while phone/PC pipelines take 100-300 ms from capture to display - enough latency to cause dizziness. For this class of device the key metric is not frame rate but end-to-end latency. This project therefore puts the whole enhancement chain inside the pixel stream, with no frame buffer at all.

Architecture

Intended main path: HDMI IN -> pixel_pack -> enhance -> pixel_unpack -> HDMI OUT, no frame buffer. Both HDMI directions are still in bring-up and are documented, not claimed: the input negotiates EDID/HPD but never locks TMDS (the stock PYNQ bitstream behaves identically), and the output stops producing TMDS once the kernel is inserted.
The enhance kernel runs local contrast -> tone curve -> binarise/invert -> split screen, all integer, one pixel per clock; 59 configuration bits are driven from the PS at run time.
Verification path: PS <-> DDR <-> AXI DMA <-> operator, readable back, so bit-exact checks need no display.
Three rotating line buffers plus per-row 3-tap shift registers give each buffer one read and one write per pixel, which is what fits II = 1 into a dual-port BRAM.

Operators

enhance: 3 BRAM, 2 DSP, 2168 FF, 1659 LUT (3%), II = 1, estimated Fmax 194.59 MHz at 7 ns. The operator family behind it - Sobel, 3x3 Gaussian, 3x3 sharpen - is hand-written HLS too (II = 1, 194.6 MHz, under 1.1k LUT each), and colour-space conversion reuses PYNQ's verified color_convert IP instead of being rewritten.

Correctness

enhance passes C simulation against an independently written C++ golden model in two configurations (contrast+split, and tone+binarise+invert+mono): 9216 beats each, 0 errors. Through the AXI-DMA path, all eight presets are compared channel by channel with the numpy model: 0 mismatches out of 2,764,800 channels each, at 15.6 ms per frame. A second pair of counters brackets the kernel and stamps the first beat in and out: the measured pipeline latency is 16 pixel clocks, i.e. 160 ns at 100 MHz - about 1% of a 720p60 line, which is what having no frame buffer buys. Border policy (horizontal replicate, vertical line-buffer state) is defined once and shared by hardware, testbench and Python model, which is why the comparison can use zero tolerance.

What Made It Work - and What Did Not

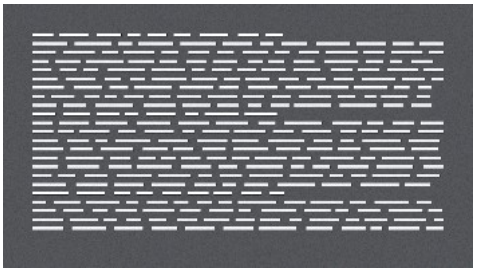
A free-running kernel (ap_ctrl_none) never drains its pipeline when the input stops, so an AXI-DMA transfer can never close; the DMA path therefore uses a bounded variant (ap_ctrl_hs plus a pixel count). The AXI DMA buffer-length register is 14 bits by default, i.e. 16 KB - not enough for a frame, so it was widened to 23 bits. The PYNQ base overlay is a 4448-line Vivado 2024.1 script; rebuilding it with 2026.1 needs exactly three patches.

Reusable PYNQ Skills and Reproducibility

Four board-independent skills ship with the project: overlay handoff linting (.hwh), bit-exact image comparison, measurement collection with PS-vs-PL table generation, and AXI-DMA transfer helpers with cache maintenance and timeouts. Each documents scope, usage, failure modes and what was verified. Everything is scripted, so a clean machine rebuilds both bitstreams with two commands.

Operator Family on the DMA Path (same board, same 1280x720 image)

Measured on the board	PS (Cortex-A9, numpy)	PL (Zynq + DMA)
enhance, 8 presets	-	15.6 ms/frame, 0/2764800 wrong
Pipeline latency	-	16 pixel clocks = 160 ns
Sobel family, frame time	**3108.4 ms**	**15.8 ms (196.7x)**



Sobel output captured from the PL path (DMA)

Tool versions: Vivado / Vitis HLS 2026.1, PYNQ 3.1.1 image, device xc7z020clg400-1. Repository: README.md, docs/ (including docs/hdmi_status.md for the open HDMI items), report/design_report.md, report/comparison.md.